

COSC 612/AIT 624

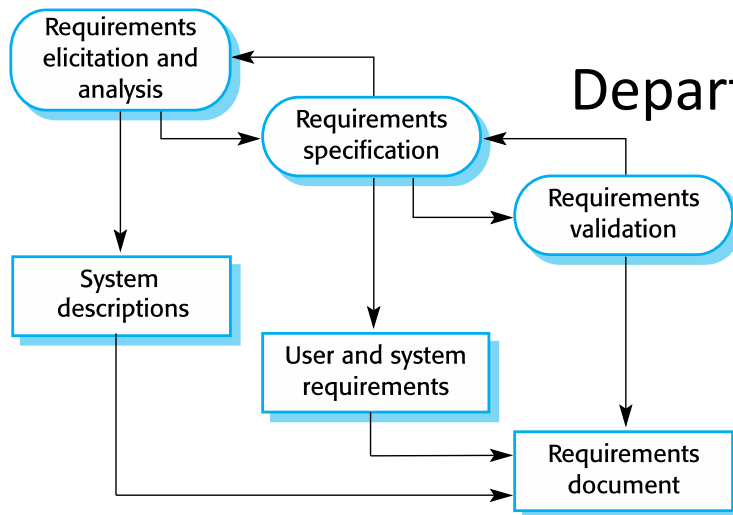
REQUIREMENTS ENGINEERING

Fall 2025

Dr. Akshita Maradapu Vera Venkata Sai

Department of Computer and Information Sciences

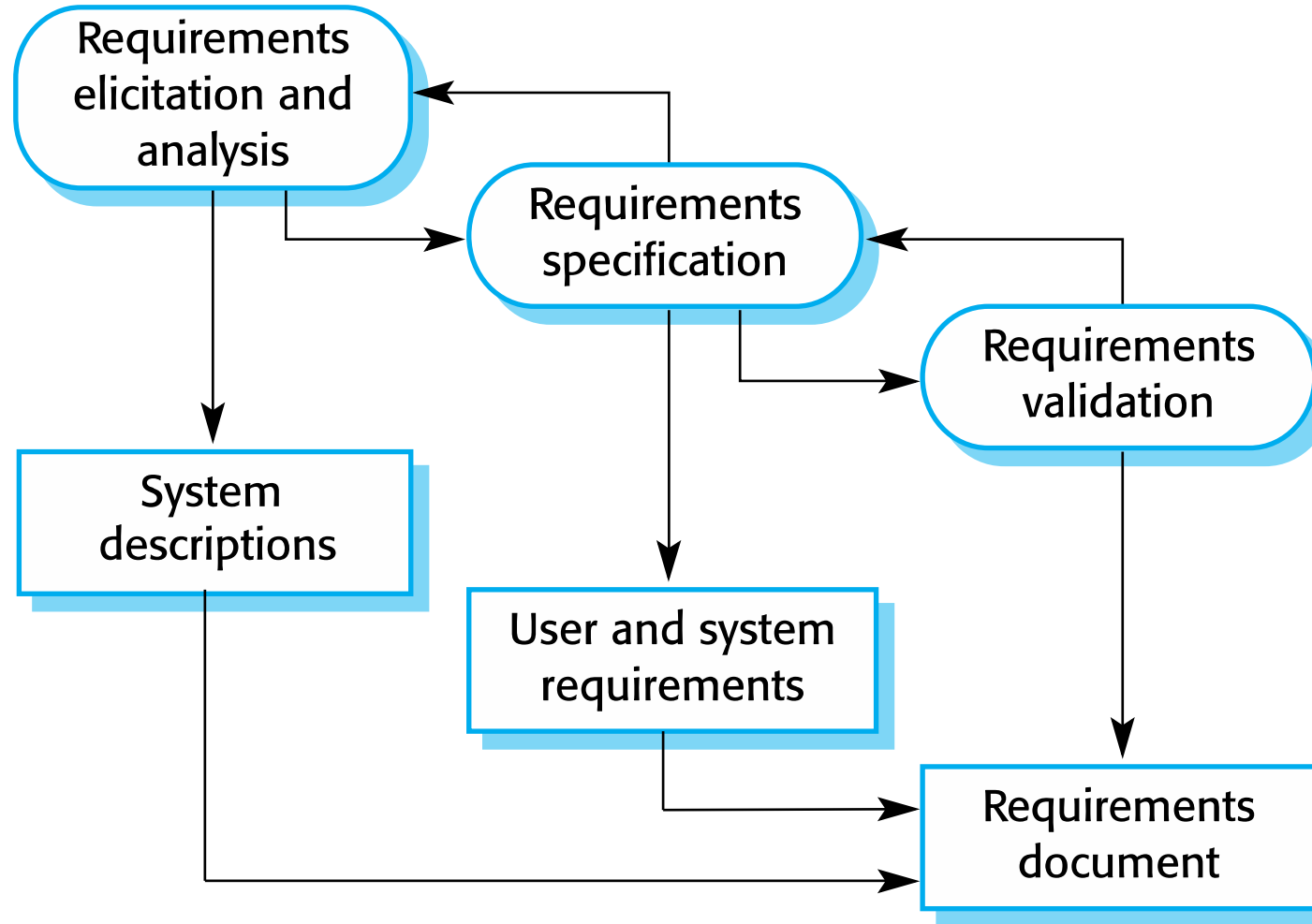
Towson University



Outlines

- Types of Requirements
- Requirements Elicitation Analysis
- Requirements Specification
- Requirements Validation
- SRS tools

Requirements Engineering Process



What is a requirement?

- A requirement is a singular documented physical or functional need that a particular product must be able to perform.
- It is a statement that identifies a necessary
 - *Attribute* (what needs to be developed ?),
 - *Capability* (what function it must perform ?),
 - *Characteristic* (how well it needs to perform ?),
 - or *quality* - **ISO 25010 software quality attributes** (e.g., usability, performance efficiency, security)
 - of a system for it to have value and utility to a stakeholder.

The statements that describe what the software system should be but not how it is to be constructed.”

Requirements Example

- *“The software shall be able to calculate the sum of a column of integers.”*
- *“The user should be able to unlock and lock the smartphone”*
 - **Attribute:** “Smart phone must have a fingerprint sensor”
 - **Capability:** “The smartphone must allow users to unlock device using fingerprint sensor”
 - **Characteristic:** “The fingerprint sensor must unlock the phone within 0.5 seconds”
 - **Quality Requirement:** “The smartphone must have a battery life of at least 12 hours on a single charge”

Types of Requirements

User Requirements:

Statements in natural language plus diagrams of the services the system provides and its operational constraints. Written for customers.

- **Usually high-level** and lacking in technical detail.
- **Gathered** from the user and form the basis for technical requirements.

System Requirements:

A structured document setting out detailed descriptions of the system's functions, services and operational constraints. Defines what should be implemented so may be part of a contract between client and contractor.

User and system requirements

Example:

User requirements definition

- 1.** The Mentcare system shall generate monthly management reports showing the cost of drugs prescribed by each clinic during that month.

System requirements specification

- 1.1** On the last working day of each month, a summary of the drugs prescribed, their cost and the prescribing clinics shall be generated.
- 1.2** The system shall generate the report for printing after 17.30 on the last working day of the month.
- 1.3** A report shall be created for each clinic and shall list the individual drug names, the total number of prescriptions, the number of doses prescribed and the total cost of the prescribed drugs.
- 1.4** If drugs are available in different dose units (e.g. 10mg, 20mg, etc) separate reports shall be created for each dose unit.
- 1.5** Access to drug cost reports shall be restricted to authorized users as listed on a management access control list.

Example

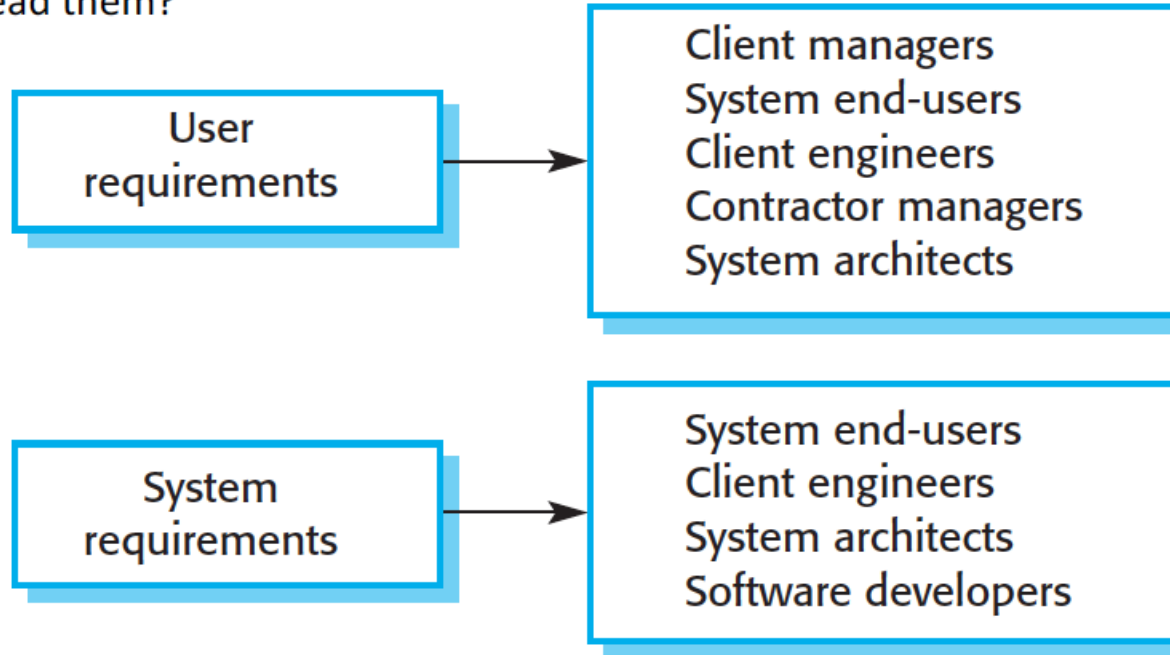
User requirement – “The user must be able to track their food delivery in real time.”

System requirement ?

User and system requirements

User vs. System Requirements

Who will read them?



Types of Requirements

- **Functional:**

- Related to the functionality of the system.
- Describe system services or actions.
- Can be used to **distinguish correct output** from incorrect.

- **Non-functional:**

- Constraints on the system or the development process.
- Related to how functionality is performed, not whether the end result is correct.

Functional Requirements

- Describe functionality or system services.
- Depend on the type of software, expected users and the type of system where the software is used.
- Functional user requirements may be high-level statements of what the system should do.
- Functional system requirements should describe the system services in detail.

Non- Functional Requirements

- These define system properties and constraints e.g. reliability, response time and storage requirements. Constraints are I/O device capability, system representations, etc.
- Process requirements may also be specified mandating a particular IDE, programming language or development method.
- Non-functional requirements may be more critical than functional requirements. If these are not met, the system may be useless.

Non-functional classifications

- **Product requirements**

- Requirements which specify that the delivered product must behave in a particular way e.g. execution speed, reliability, etc.

- **Organisational requirements**

- Requirements which are a consequence of organisational policies and procedures e.g. process standards used, implementation requirements, etc.

- **External requirements**

- Requirements which arise from factors which are external to the system and its development process e.g. interoperability requirements, legislative requirements, etc.

Non- Functional Requirements

Examples:

Product Requirement:

The system shall be available to all clinics during normal working hours (Mon–Fri, 0830–17.30). Downtime within normal working hours shall not exceed five seconds in any one day.

Organizational Requirement:

Users of the system shall authenticate themselves using their health authority identity card.

External requirement:

The system shall implement patient privacy provisions as set out in HStan-03-2006-priv.

Activity – Functional and Non-functional requirements

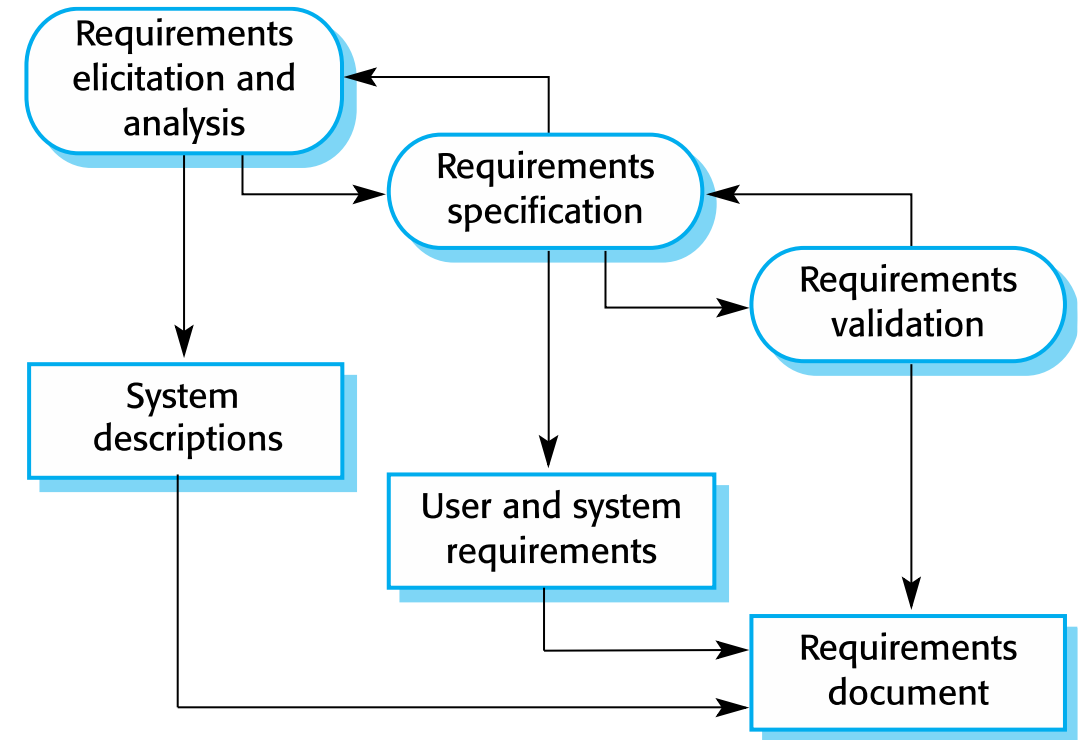
Imagine you are developing a **food delivery app** with the following requirements:

1. The app must allow users to create an account and log in.
2. The app must load the restaurant list within 3 seconds.
3. Users must be able to add and remove items from their cart before placing an order.
4. The app must be available 99.9% of the time.
5. Users must be able to make payments using credit cards, PayPal, and Apple Pay.
6. The app must use AES-256 encryption for all payment transactions.
7. The app must have a simple and intuitive user interface.
8. Users must receive real-time updates on their order status.
9. The app should handle up to 10,000 orders per minute without crashing.
10. Restaurants must be able to update their menus and prices through a web portal.

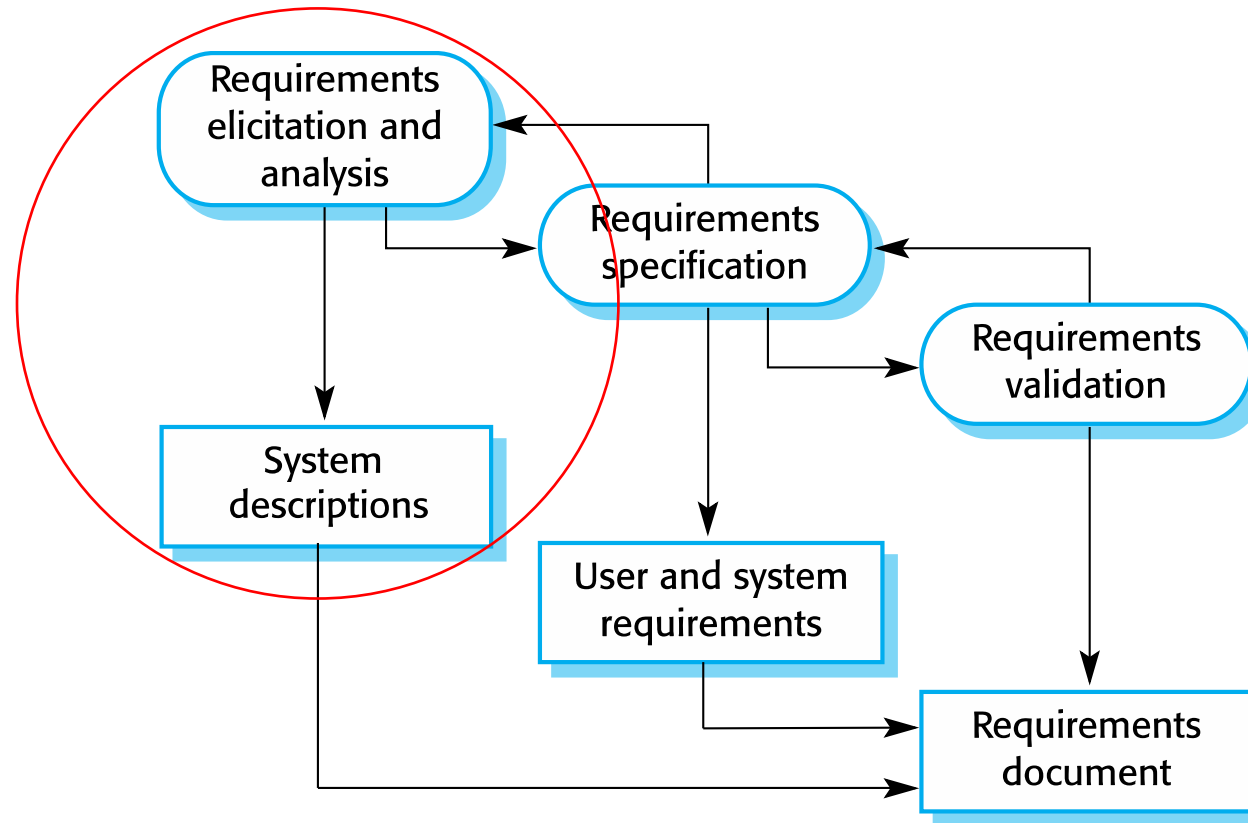
Requirements Engineering (RE)

Requirements Engineering Process

- Requirements engineering process?
 - *Requirements elicitation and analysis*
 - What do the system stakeholders require
 - or expect from the system?
 - *Requirements specification*
 - Defining the requirements in detail
 - *Requirements validation*
 - Checking the validity of the requirements



Requirements Elicitation



Requirements Elicitation

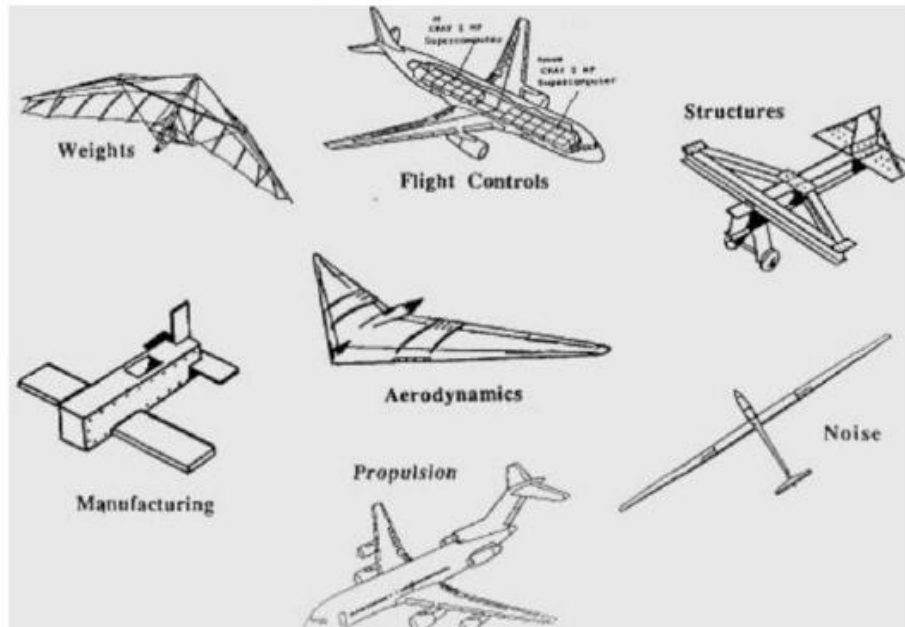
- The process of working with customers to learn about the application domain, the services that the system should provide, and the system's operational constraints.
- Involves all stakeholders:
 - end-users, managers, maintenance team, domain experts, trade unions, lawyers, etc...

The Stakeholders

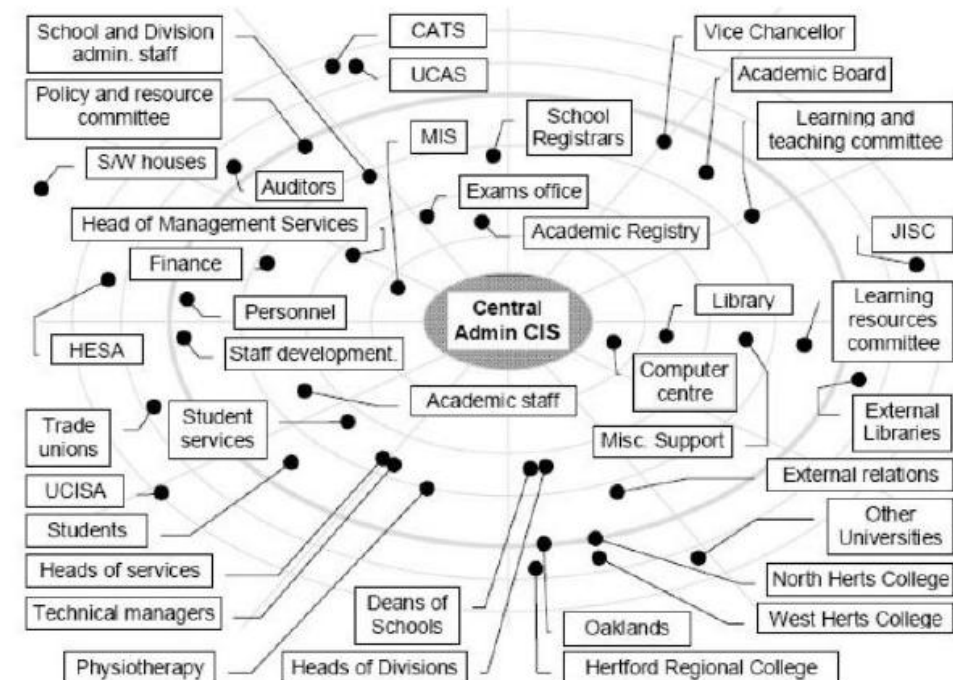
- The **specification process** must involve all **stakeholders**:
 - Clients
 - Engineers
 - Regulatory Agencies
 - Users
- **Do all of these want the same thing?**

Requirements Elicitation

- **One requirement** can have many meanings
 - depending on **the stakeholder's perspective**



- There can be many stakeholders



Requirements Elicitation – Issue?

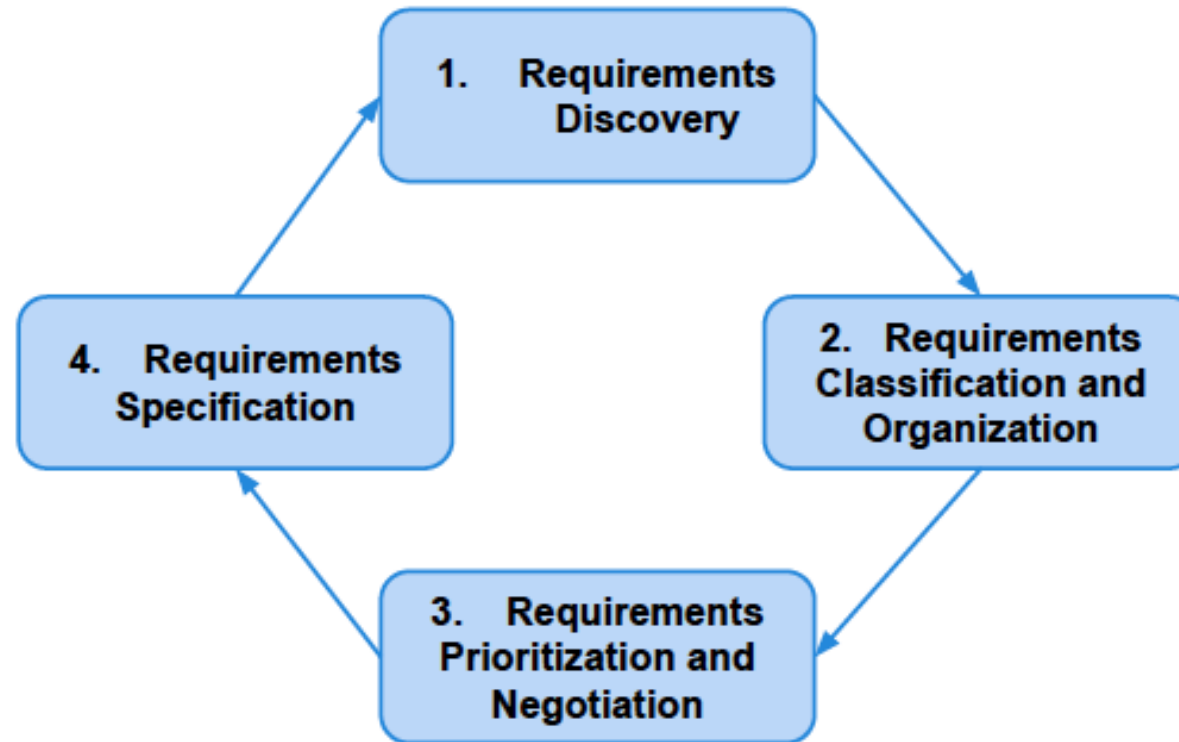


Requirements Elicitation– Issue?

Very Difficult!!!! But Why ??

- Stakeholders don't know what they want from a computer system except in the most general terms.
- Stakeholders express requirements with implicit knowledge of their domain.
- Different stakeholders have different requirements.
- Economic and business environment is dynamic during elicitation.

Requirements Elicitation Process



Requirements Elicitation

Stakeholders Must Work Too



Requirements Elicitation

• Interview the Stakeholder

- Make sure you have the right answer.
- Make sure this is the official answer.
 - If an answer is unclear, keep asking questions.
 - If somebody else tells you a different answer, politely ask for clarification.
- Find out what they are willing to pay for each function (helps in prioritization).
 - If unwilling/unable to place a \$ amount, ask how important it is for their daily work.

These two
often conflict!

Requirements Elicitation

- **Effective Interviewing**

- Try not to alienate the stakeholder:
 - Avoid “We thought you knew that.” and “We always do it that way.”
- Hundreds of techniques
 - Most important: do your homework - research a problem before the interview.
 - Be polite, but firm - keep asking until you feel you could deliver a working function.
 - Follow up after writing requirements down.

Requirements Elicitation

- **Interview the Stakeholder:** Types of Viewpoint

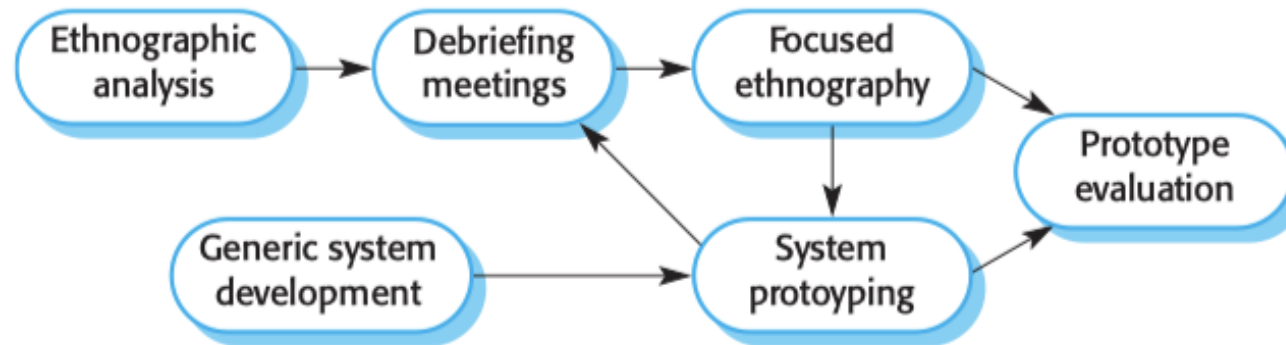
- **Receivers of services**
 - People or systems that receive services from your system.
- **Data sources or sinks**
 - What kind of data is produced or consumed by the stakeholders and system?
- **Experts in the domain**
 - Tend to notice details that novices will miss.

Requirements Elicitation

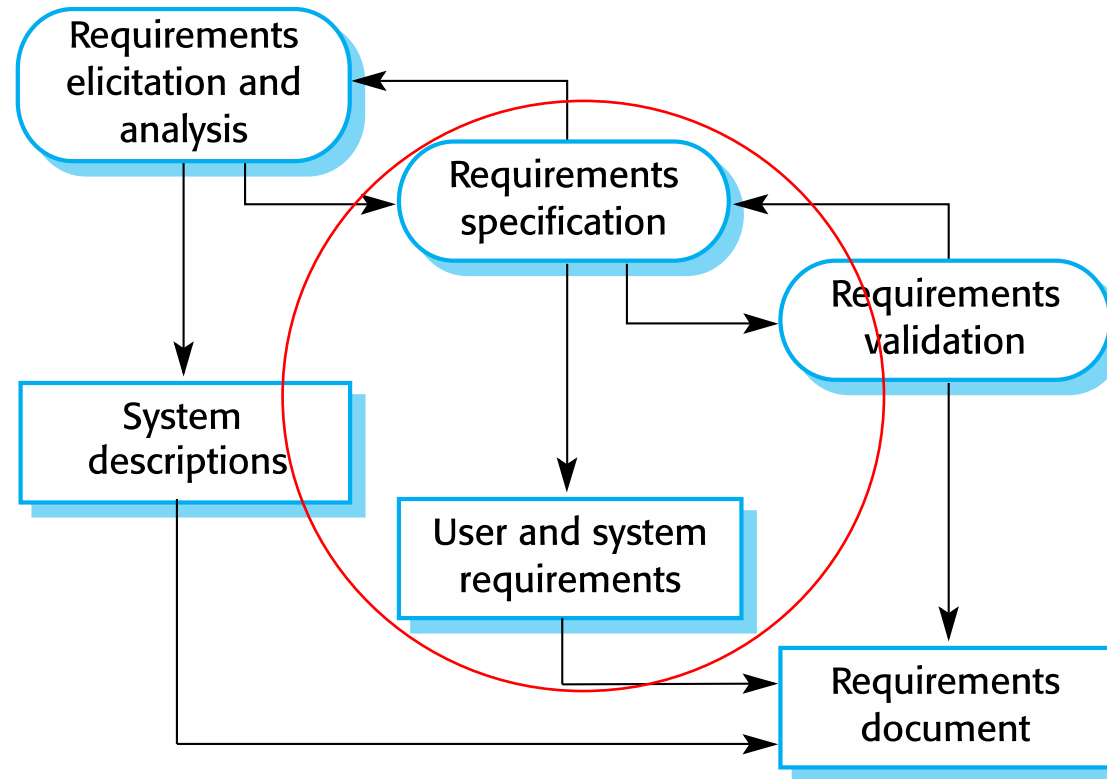
Ethnography

Qualitative research to understand how people use technology in their natural environment

Ex : Market studies, focus groups



Requirements Specifications



Natural Language Specification

Normal Method:

Natural language statements, supplemented by diagrams and tables.

This is universally understandable, but problems can arise.

What are some of the problems with natural language requirements?

- **Lack of clarity**
 - Precision is difficult without making the document hard to read, and it is hard to write unambiguously.
- **Missing details**
 - It is easy to forget to include details when using natural language.
- **Requirements amalgamation**
 - Several different requirements may be expressed together.

Writing Requirements

Level of Details

The level of detail of requirements may range from a high-level abstract statement to a detailed mathematical functional specification.

This is inevitable: requirements serve dual functions:

- May be the basis for a bid for a contract
 - Therefore, open to interpretation.
- May be the basis for the contract itself
 - Therefore, must be defined in detail.

Capturing Good Requirements

Three common problems:

- Poorly written individual requirements
 - Ambiguous, contradictory, unclear.
- Untestable requirements
 - Cannot verify that the final system meets the requirements, as written.
- Poorly structured requirements documents
 - Unable to find necessary information.

Structured Specification

- Standard way that retains the expressiveness of natural language specification
- Ensures uniformity over all the requirements
- In order to ensure uniformity – a template is used to specify requirements.
- Two approaches:
 - Structured Template - **Lists requirements** (what the system must do).
 - Use case template - **Describes interactions** (step-by-step user behavior & system responses).

Structured Specification

- **Number:** A unique identifier.
- **Use Case:** Which use-case is this requirement linked to.
- **Introduction/Definition:** What is this requirement about?
- **Rationale:** Why does the requirement exist?
- **Source:** Who came up with the requirement?
- **Author:** Who wrote it down?
- **Inputs:** What are the necessary inputs to use this function?
- **Required Function:** What is the requirement?
- **Outputs:** What is output as a result of this function?
- **Related Requirements:** List of relevant requirements?
- **Conflicts:** Are there requirements in conflict with this one?
- **Support Material:** Docs, figures, tables, etc.
- **Test Cases:** How do we test this requirement?
- **Date:** When was this requirement modified last?
- **Priority:** How important is this requirement?

Suggested Template Items : Example

Introduction: The most common action performed at an ATM is the withdrawal of funds. The ATM must support this functionality.

Rationale: Survey ABC-345 indicates that withdrawals are highly desirable. The success of the product hinges on successful withdrawal of funds.

Inputs: Card number, PIN, requested amount

Description: The user shall be able to withdraw funds from the ATM

- A withdrawal shall be allowed if and only if:
 - The Card Number can be validated (Req 35)
 - The PIN is valid for the card (Req 45)
 - The funds in the card account exceeds the Requested Amount requested in the withdrawal
- If a withdrawal is allowed, the Requested Amount of money shall be dispensed.

Outputs: Customer Receipt, Requested Amount of Money

Persistent Changes: The Requested Amount will be deducted from the Customer Account.

Related Requirements: 35, 45

What is a Use Case?

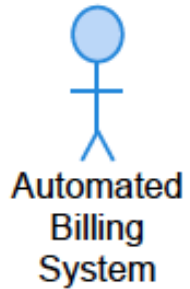
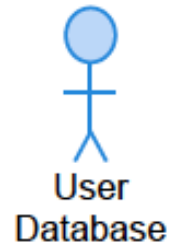
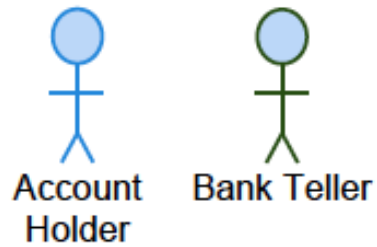
A **use case** captures some visible **function** of the system.

- A use case is a **goal** that an actor can accomplish using a system, through a series of **user interactions**.
 - Transfer funds.
 - Query balance.
- This may be a large or small function.
 - Depends on the chosen level of detail.
 - Withdraw Funds vs Validate PIN

Use Cases

- Accompanied by a **use case description** detailing the user interactions required to accomplish the use case.
- **Acquired through interviews** with stakeholders and viewpoint analysis.
- Useful for **eliciting** and **refining requirements**.

Use Case: What is an actor



An **actor** is a role a user plays with respect to the system.

- Actors carry out use cases. An actor can perform many use cases. A use case can involve multiple actors.
- A single user can be multiple actors, depending on how they use a system.
- Actors do not need to be human - can be an external system (hardware or software) that interacts with the system being built.

Use Case Template

- **Identifier:** Unique ID number
- **Iteration:** Version number
- **Summary:** User goal being fulfilled
- **Actors:** What users/databases/external systems are involved?
- **Basic Course of Events:** Sequence of user interactions.
- **Alternative Paths:** Alternate sequences of events that stem off from certain points.
- **Exception Paths:** Error sequences that stem off from certain points.
- **Extension Points:** Use-cases that resume from the end of this use-case.
- **Trigger:** Rationale, what causes this interaction sequence to begin.
- **Assumptions:** Constraints assumed on this use-case.
- **Precondition:** Conditions that must hold for this use-case to take place, may list other use-cases that need to be completed first.
- **Postcondition:** Side-effects of this use-case.
- **Author:** Who wrote this use-case.
- **Date:** When was this last modified?

Withdraw Cash (In Sample Template)

- **Summary:** The customer requests cash and the ATM dispenses the cash.
- **Basic Course of Events:**
 - 1. Completion of use case *Validate PIN*.
 - 2. The customer selects the withdrawal menu option.
 - 3. The ATM asks the customer for the account from which to withdraw the cash.
 - ...
 - 9. If there are sufficient funds, the cash is dispensed and the amount is withdrawn from the account.
 - 10. Complete use case *Complete Transaction*.
- **Alternative Paths:** In steps 4, 6, and 8, the customer can cancel the transaction and go directly to step 10. If the customer does not confirm the account in step 8, proceed directly to step 10.
- **Exception Paths:** In step 9, if there are not sufficient funds, then an error message is displayed and execution proceeds to step 10.
- **Precondition:** The Validate PIN use case completed successfully.
- **Postcondition:** The cash is dispatched and the amount has been withdrawn from the selected account.

When should we use Use Cases?

- In short... Always!
- Requirements specification is the hardest part of software development. Use cases are a powerful tool to understand:
 - Who your users are (including non-human systems).
 - What functions your system should provide.
 - How these functions work (at a high level).

Use Cases: Remember

- Each use case will likely correspond to many requirements. Use cases are high level goals, requirements are low level statements of how to make that goal achievable.
- Use cases represent an external view of the system. They do not tell you what your system objects are, and should not feature internal objects as actors.
- No “rule of thumb” for how many use cases you should have:

Ask yourself: does this capture all of the goals user might have when using my system?

Improving Requirements

The product shall switch between displaying and hiding non-printing characters instantaneously.

Problems:

- Instantaneously is not testable - subjective.
- When is it switching? What causes this switch? Is it random?

Improving Requirements

Old

The product shall switch between displaying and hiding non-printing characters instantaneously.

New

The text entry box shall switch between displaying and hiding non-printing characters within 200ms of mouse release of the display button in the quick function bar (Req 3.4).

Improving Requirements – Incomplete requirement

The system shall respond to all user requests within 2 seconds.

Problems:

- Too overarching - need to address different user requests one at a time, each might have different needs.
- Very likely to lead to inconsistent requirements.

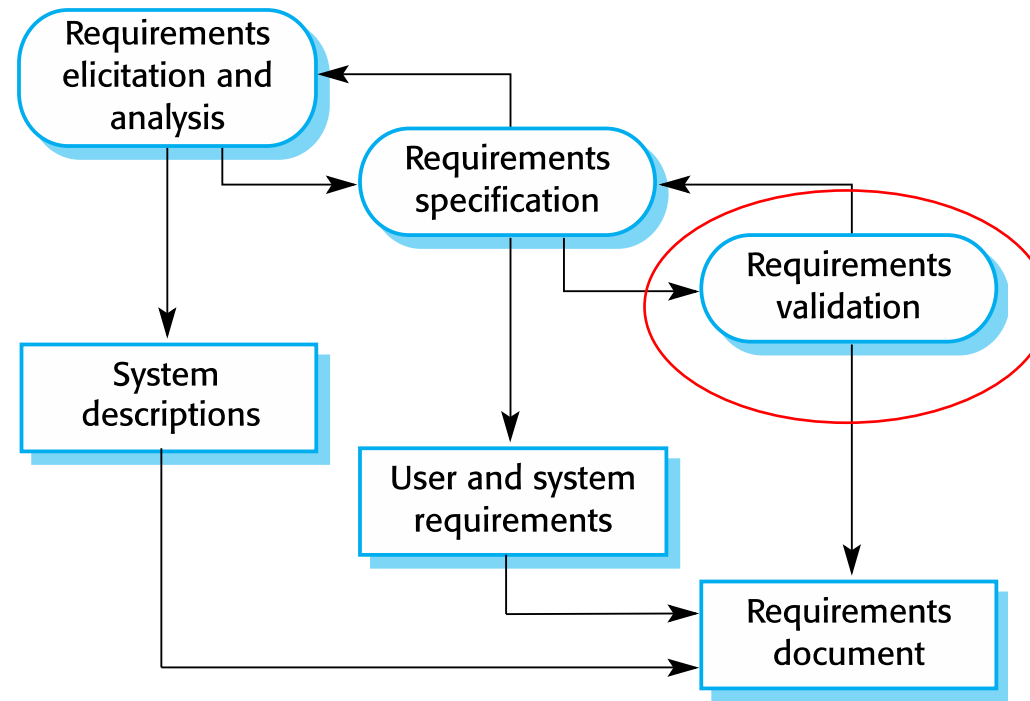
Requirements Specification Document

Chapter	Description
Preface	This defines the expected readership of the document and describe its version history, including a rationale for the creation of a new version and a summary of the changes made in each version.
Introduction	This describes the need for the system. It should briefly describe the system's functions and explain how it will work with other systems. It should also describe how the system fits into the overall business or strategic objectives of the organization commissioning the software.
Glossary	This defines the technical terms used in the document. You should not make assumptions about the experience or expertise of the reader.
User requirements definition	Here, you describe the services provided for the user. The nonfunctional system requirements should also be described in this section. This description may use natural language, diagrams, or other notations that are understandable to customers. Product and process standards that must be followed should be specified.
System architecture	This chapter presents a high-level overview of the anticipated system architecture, showing the distribution of functions across system modules. Architectural components that are reused should be highlighted.

Requirements Specification Documents

System requirements specification	This describes the functional and nonfunctional requirements in more detail. If necessary, further detail may also be added to the nonfunctional requirements. Interfaces to other systems may be defined.
System models	This chapter includes graphical system models showing the relationships between the system components and the system and its environment. Examples of possible models are object models, data-flow models, or semantic data models.
System evolution	This describes the fundamental assumptions on which the system is based, and any anticipated changes due to hardware evolution, changing user needs, and so on. This section is useful for system designers as it may help them avoid design decisions that would constrain likely future changes to the system.
Appendices	These provide detailed, specific information that is related to the application being developed—for example, hardware and database descriptions. Hardware requirements define the minimal and optimal configurations for the system. Database requirements define the logical organization of the data used by the system and the relationships between data.
Index	Several indexes to the document may be included. As well as a normal alphabetic index, there may be an index of diagrams, an index of functions, and so on.

Requirements Validation



Requirements Validation

- What is requirements validation?
 - Finding errors in requirements
 - Checking if the requirements define the system that the customer wants
- Overlaps with requirements elicitation and analysis
- Why should we validate requirements?
 - If not done properly, errors will be discovered at a later stage
 - Can lead to expensive re-work

How to go about with validation?

- Requirements validation can be done :
 - By carrying out different checks on the requirements document
 - By using different validation techniques

Validation Checks

1. Validity checks

- This checks if the requirements reflect the real need of customers

2. Consistency checks

- To see if there are no conflicts in the requirements
 - The requirements must not have contradictory constraints or,
 - Have different descriptions for the same requirements

Validation Checks

3. Completeness checks

- Checks if all the required information is provided for each requirement
- This includes constraints, dependencies and function definitions

4. Realism Checks

- To check if the stated requirements can be met :
 - Under the budget and schedule constraints
 - If we have the tool support to realize these requirements

5. Verifiability

- This is done to avoid disputes between the customer and contractor
- How?
 - Have some automated tests that demonstrates that the system built meets the specified requirements

Validation Techniques

1. Requirements reviews

- The requirements are analyzed systematically by a team of reviewers who check for errors and inconsistencies.

2. Prototyping

- Creating an executable model to provide to the customer
- Collect the customer feedback and send the changed requirements to the developers

3. Test-case generation

- Requirements should be testable.
- To reveal requirements problems.
- If a test is difficult or impossible to design:
 - the requirements will be difficult to implement and should be reconsidered.
- Developing tests from the user requirements before any code is written is an integral part of test-driven development.

Test-First Development and Test Case Generation

Test Plan

- Plan for how we will test the system.
 - **What** is being tested (units of code, features).
 - **When** it will be tested (required stage of completion).
 - **How** it will be tested (what scenarios do we run?).
 - **Where** we are testing it (types of environments).
 - **Why** we are testing it (what purpose does this test serve?).
 - **Who** will be responsible for writing test cases (assign responsibility).

What Goes Into a Test?

- The anatomy of a test case
 - **Inputs** (test data) to the system.
 - Predicted **outputs** based on these inputs.
 - **Procedure** needed to exercise the system.
 - Pre-conditions and set-up steps.
 - Things that we will need to do to gather data.
 - How the actual output will be compared to the expected output.

Requirements-Based Testing

- Process of deriving tests from the requirement specifications.
 - Typically the baseline technique for designing test cases. Can begin as part of requirements specification, and continue through each level of design and implementation.
 - Basis of verification - builds evidence that the implementation conforms to its specification.
 - Effective at finding some classes of faults that elude code-based techniques.
 - i.e., incorrect outcomes and missing functionality

Typical Requirements

- After a high temperature is detected, an alarm must be raised quickly.
- Novice users should be able to learn the interface with little training.

How do we make these requirements testable?

Test the Requirement

After a high temperature is detected, an alarm must be raised quickly.

Test Case 1:

- Input:
 - Artificially raise the temperature above the high temperature threshold.
- Procedure:
 - Measure the time it takes for the alarm to come on.
- Expected Output:
 - The alarm shall be on within 2 seconds.

Test the Requirement

Novice users should be able to learn the interface with little training.

Test Case 2:

- Input:
 - Identify 10 new users and put them through the training course (maximum length of 6 hours)
- Procedure:
 - Monitor the work of the users for 10 days after the training has been completed
- Expected Output:
 - The average error rate over the 10 days shall be less than 3 entry errors per 68 hours of work.

How Many Tests Do You Need?

Testing a requirement does not mean writing a single test.

- You normally have to write several tests to ensure that the requirement holds.
 - What are the different conditions that the requirement must hold under?
- Maintain traceability links from tests to the requirements they cover.

Deriving tests - Scenario Testing

One method of deriving tests is to use **scenarios** to develop test cases for the system.

- Stories that describe one way in which a system might be used.
 - Use case descriptions, user stories, sequences of user interactions.
- Stories should be complex and credible.
- Should be easy to evaluate.

Scenario Example

For the patient management system:

Kate is a nurse. One of her responsibilities is to visit patients at home to check on the progress of their treatment. On a day for home visits, Kate logs into the PMS and uses it to print her schedule of home visits for that day, along with summary information about the patients to be visited. She requests that the records for these patients be downloaded to her tablet. She is prompted for her password to encrypt the records for the tablet.

One of the patients, Jim, is being treated for depression. Jim feels that the medicine is keeping him awake at night. Kate looks up Jim's record and is prompted for her key phrase to decrypt the record. She checks the drug prescribed and queries its side effects. She notes the problem in Jim's record and enters a prompt to call him when she gets back to the office to schedule an appointment with a physician. The system re-encrypts Jim's record.

After finishing her consultations, Kate uploads her records to the database. The system generates a call list for Kate of those patients who need to schedule a follow-up appointment.

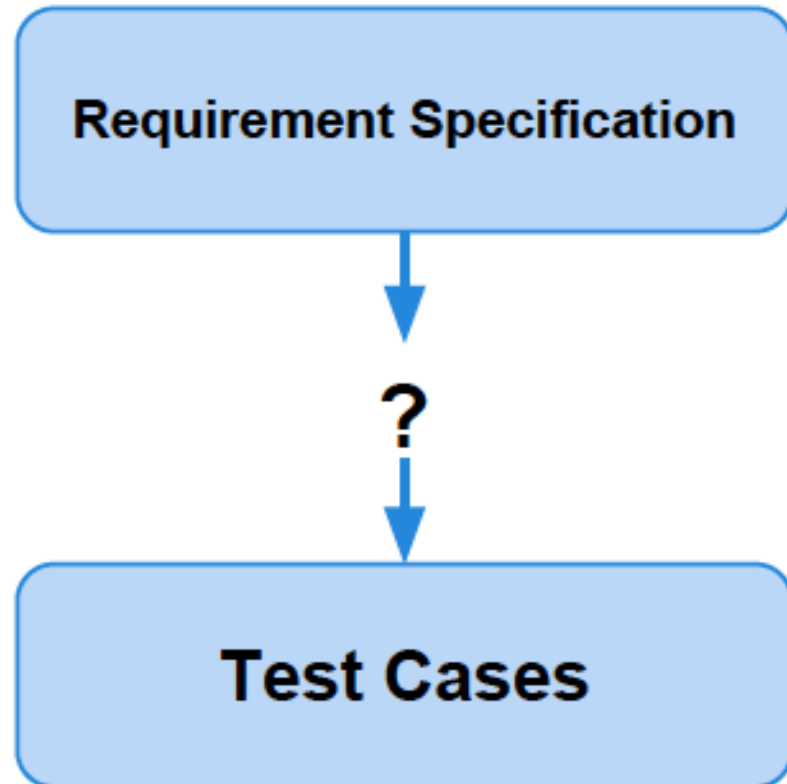
Features Tested?

Patient System - Features Tested

This single scenario would test:

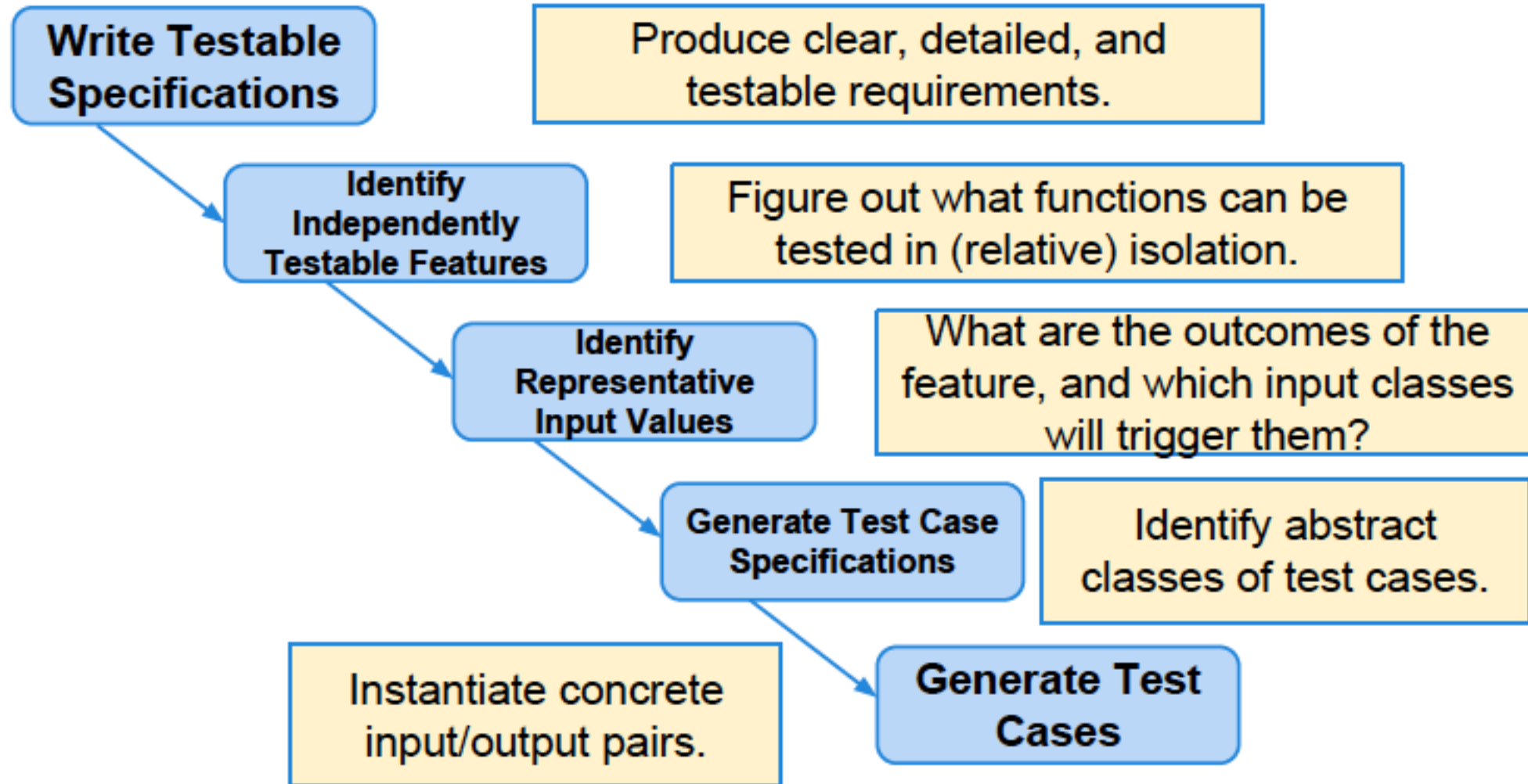
- Authentication
- Downloading to a mobile device and uploading changes
- Home visit scheduling
- Encryption and decryption of patient records on a mobile device
- Record retrieval and modification
- Links with drug database
- System for call prompting

A Model of Testing



- Where we're at:
 - "Set up a patient record with no known allergies. Prescribe medication for allergies that are known to exist. Check that a warning message is not issued by the system."
 - Generic scenarios that can be used as the basis for test cases.
- We need concrete test cases that can be run.

Creating Requirements-Based Tests



Step 1 : Independently Testable Feature

- Requirements are difficult to test in isolation. However, the system can usually be decomposed into the functions it provides.
- **An independently testable feature is a well-defined function that can be tested in (relative) isolation.**
- Identified to “divide and conquer” the complexity of functionality.

This single scenario would test:

- Authentication
- Downloading to a mobile device and uploading changes
- Home visit scheduling
- Encryption and decryption of patient records on a mobile device
- Record retrieval and modification
- Links with drug database
- System for call prompting

Step 2: Features and Parameters

Tests for features must be described in terms of all of the **parameters** and **environmental factors** that influence the feature's execution.

- What are the inputs to that feature?
 - User registration on a website might take in:
 - `(firstName, lastName, dateOfBirth, eMail)`
- Consider implicit environmental factors.
 - Registration also requires a user database.
 - The existence and contents of that database influence execution.

Parameter Characteristics

The key to identifying tests is in understanding *how the parameters are used by the feature.*

- **Type** information is helpful.
 - `firstName` is a string, the database contains `UserRecord` structs.
- ... but **context** is important.
 - If the database already contains an entry for that combination of fields, registration should be rejected.
 - `dateOfBirth` is a collection of three integers, but those integers are not used for any arithmetic operations.

Example Step 1: Features

Class Registration System

What are some independently testable features?

- Add class
- Drop class
- Modify grading scale
- Change number of credits
- Graphical interface of registration page

Remember

environmental factors

(**Parameter Characteristics, ex. type and context**)

that influence

features' execution

Example Step 2: Parameters

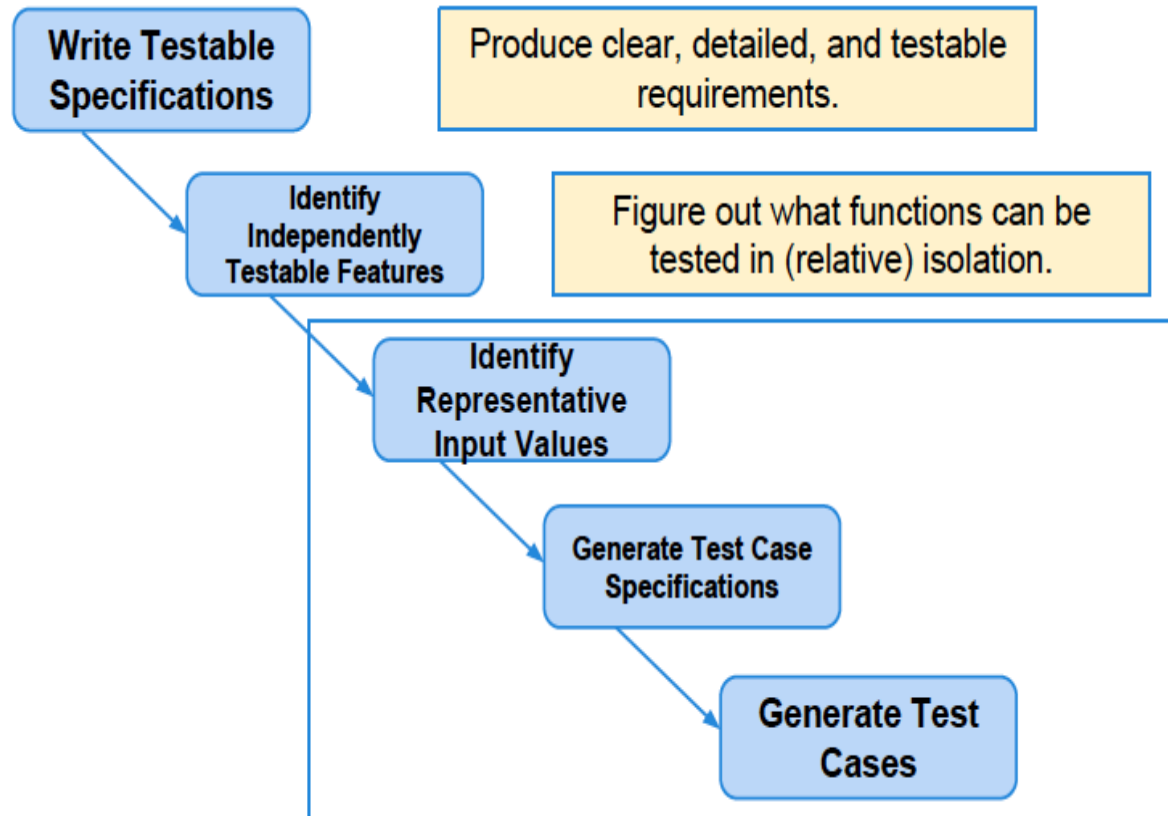
Adding a class

What are the **parameters**?

- Course number to add
- Grading basis
- Student record
- What about a course database? Student record database?

Remember
environmental factors
(**Parameter Characteristics, ex. type and context**)
that influence
features' execution

Where are we now and what is next?

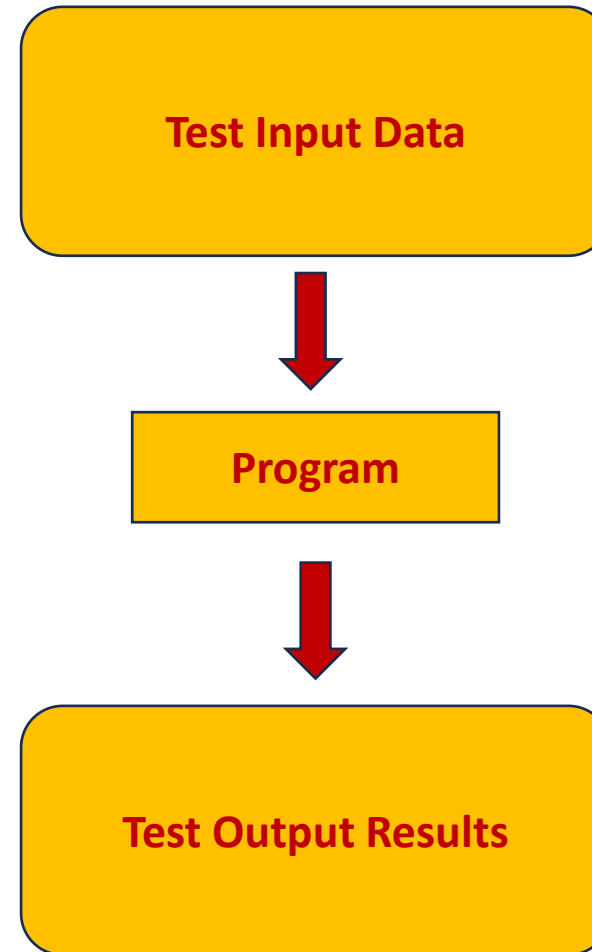


develop test cases for each requirement.

- If the requirement cannot be tested, you **most likely have a bad requirement.**
 - Rewrite it so it is testable.
 - Remove the requirement if it can't be rewritten.
 - Point out why it is an unstable requirement.
- Your requirements and testing effort will be greatly improved!

Step 3: Identifying Representative Values

- We know the features. We know their parameters.
- What input values should we pick?
- **What about exhaustively trying all inputs?**



Exhaustive Testing

Take the arithmetic function for the calculator:

```
add(int a, int b)
```

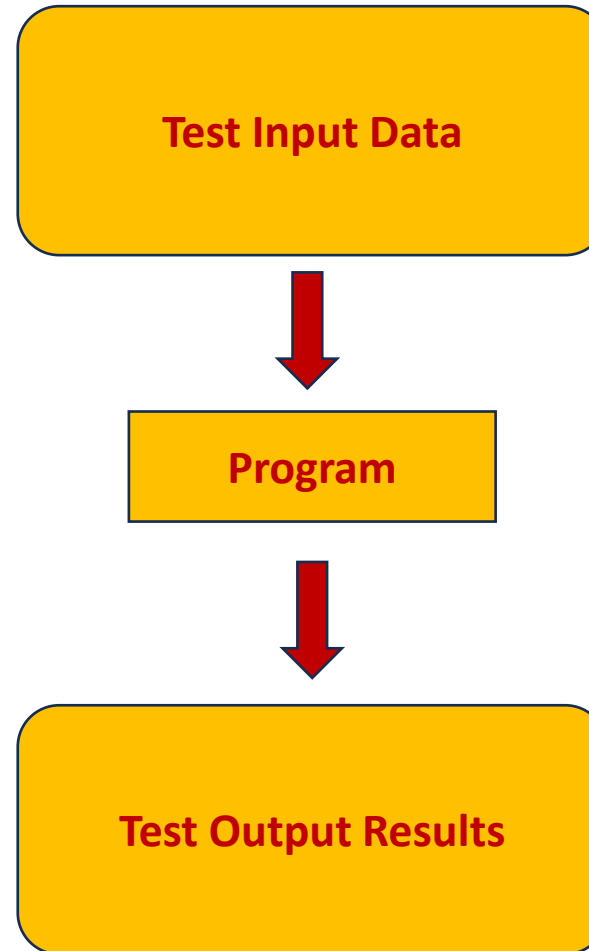
- How long would it take to exhaustively test this function?

2^{32} possible integer values for each parameter.
 $= 2^{32} \times 2^{32} = 2^{64}$ combinations
 $= 10^{13}$ tests

1 test per nanosecond
 $= 10^5$ tests per second
 $= 10^{10}$ seconds
Or... About 600 years!

Not all Inputs are Created Equal

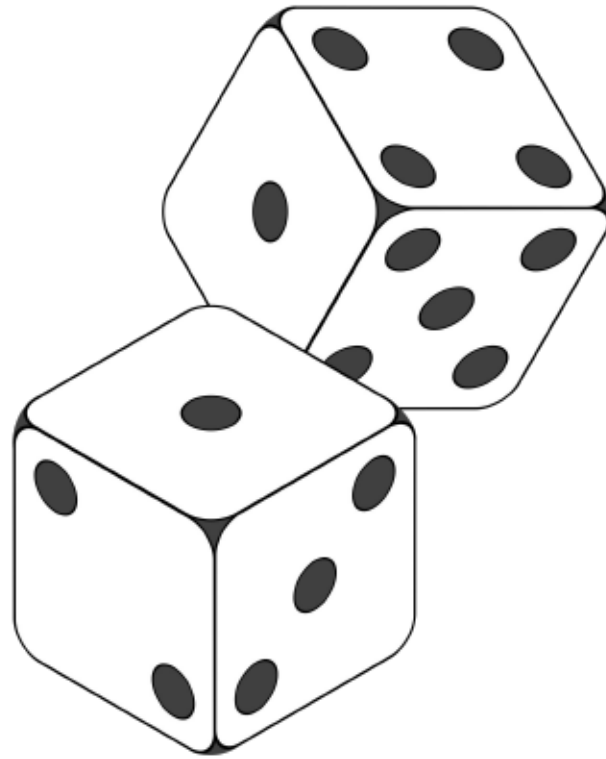
- We can't exhaustively test any real program.
 - **We don't need to!**
- Some inputs are better than others at revealing faults, but we can't know which in advance.
- Tests with different input than others are better than tests with similar input.



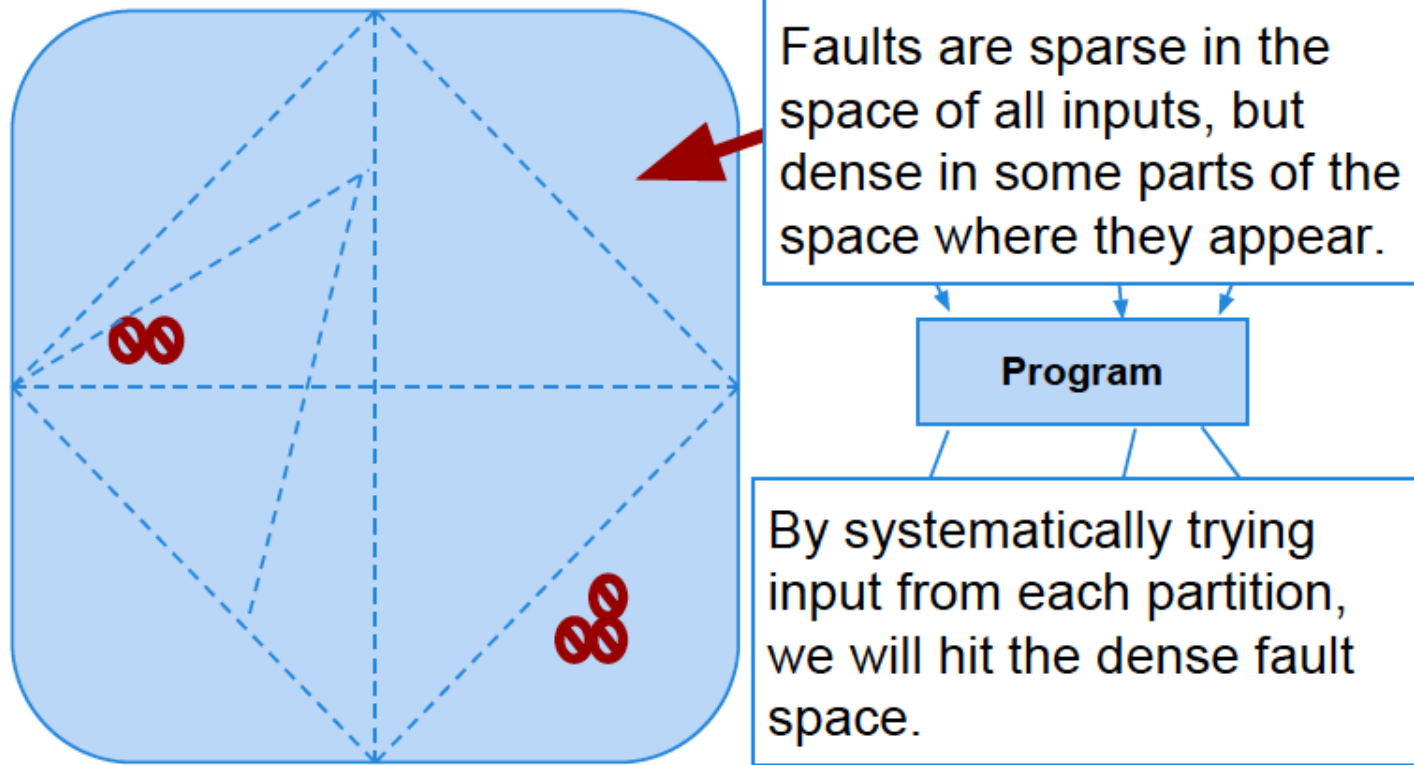
**How to
pick
inputs?**

Method 1: Random Testing

- Pick inputs uniformly from the distribution of all inputs.
- All inputs considered equal.
- Keep trying until you run out of time.
- No designer bias.
- Removes manual tedium.



Method 2: Input Partitioning



Method 3: Equivalence Class

- We want to divide the input domain into *equivalence classes*.
 - Inputs from a group can be treated as the same thing (trigger the same outcome, result in the same behavior, etc.).
 - If one test reveals a fault, others in this class (probably) will too. In one test does not reveal a fault, the other ones (probably) will not either.
- Perfect partitioning is difficult, so grouping based largely on intuition, experience, and common sense.

Input partitioning and equivalence classes are closely related and often used together

How?

Examples

```
substr(string str, int index)
```

What are some possible partitions?

Parameter	Partitions
Str	Empty String “ ”
	Single character string “a”
	String with several characters
Index	0
	< 0 [works only in a few languages]
	$0 \leq \text{index} < \text{str.length} - 1$
	$\geq \text{str.length}$

Equivalence classes

- What are possible equivalence classes

Parameter	Partitions	Equivalence classes
Str	Empty String “ ”	“ ”
	Single character string	Any single character
	String with several characters	Strings of lengths 4, 8, 10 , etc.
Index	0	0
	< 0	[-inf, -1]
	0 <= index < str.length -1	[0, (str.length/2)-1]; str.length/2; [(str.length/2)+1, str.length-1]
	> = str.length	[str.length, +inf]

Choosing Input Partitions

- Look for equivalent output events.
- Look for ranges of numbers or values.
- Look for membership in a logical group.
- Look for time-dependent equivalence classes.
- Look for equivalent operating environments.
- Look at the data structures involved.
- Remember invalid inputs and boundary conditions.

Look for Equivalent Outcomes

- It is often easier to find good tests by looking at the outputs and working backwards.
 - Look at the outcomes of a feature and group input by the outcomes they trigger.
 - Example: A graphics routine that draws lines on a canvas. Outcomes include:
 - No line
 - Thin, short line
 - Thin, long line
 - Thick, short line
 - ... etc.
- Look for equivalent output events.
 - Look for ranges of numbers or values.
 - Look for membership in a logical group.
 - Look for time-dependent equivalence classes.
 - Look for equivalent operating environments.
 - Look at the data structures involved.
 - Remember invalid inputs and boundary conditions.

Look for Ranges of Values

- If an input is intended to be a 5-digit integer between 10000 and 99999, you want partitions:

<10000, 10000-99999, >100000

- Other options: < 0, max int, real-valued numbers
- You may want to consider non-numeric values as a special partition.

- Look for equivalent output events.
- Look for ranges of numbers or values.
- Look for membership in a logical group.
- Look for time-dependent equivalence classes.
- Look for equivalent operating environments.
- Look at the data structures involved.
- Remember invalid inputs and boundary conditions.

Timing Partitions

The timing and duration of an input may be as important as the value of the input.

- Very hard and very crucial to get right.
 - Trigger an electrical pulse 5ms before a deadline, 1ms before the deadline, exactly at the deadline, and 1ms after the deadline.
 - Push the “Esc” key before, during, and after the program is writing to (or reading from) a disc.
- Look for equivalent output events.
 - Look for ranges of numbers or values.
 - Look for membership in a logical group.
 - Look for time-dependent equivalence classes.
 - Look for equivalent operating environments.
 - Look at the data structures involved.
 - Remember invalid inputs and boundary conditions.

Equivalent Operating Environments

- The environment may affect the behavior of the program. Thus, environmental factors can be partitioned and varied when testing.
- Available memory may affect the program.
- Processor speed and architecture.
 - Try with different machine specs.
- Client-Server Environment
 - No clients, some clients, many clients
 - Network latency
 - Protocols (SSH vs FTP, HTTP vs HTTPS)

- Look for equivalent output events.
- Look for ranges of numbers or values.
- Look for membership in a logical group.
- Look for time-dependent equivalence classes.
- Look for equivalent operating environments.
- Look at the data structures involved.
- Remember invalid inputs and boundary conditions.

Data Structure Can Suggest Partitions

Certain data structures are prone to certain types of errors. Use those to suggest equivalence classes.

For sequences, arrays, or lists:

- Sequences that have only a single value.
- Different sequences of different sizes.
- Derive tests so the first, middle, and last elements of the sequence are accessed.

- Look for equivalent output events.
- Look for ranges of numbers or values.
- Look for membership in a logical group.
- Look for time-dependent equivalence classes.
- Look for equivalent operating environments.
- Look at the data structures involved.
- Remember invalid inputs and boundary conditions.

Do Not Forget Invalid Inputs!

- Likely to cause problems. Do not forget to incorporate them as input partitions.
 - Exception handling is a well-known problem area.
 - People tend to think about what the program should do, not what it should protect itself against.
- Take these into account with all of the other selection criteria already discussed.

- Look for equivalent output events.
- Look for ranges of numbers or values.
- Look for membership in a logical group.
- Look for time-dependent equivalence classes.
- Look for equivalent operating environments.
- Look at the data structures involved.
- Remember invalid inputs and boundary conditions.

Input Partition Example

What are the input partitions for:

`max(int a, int b) returns (int c)`

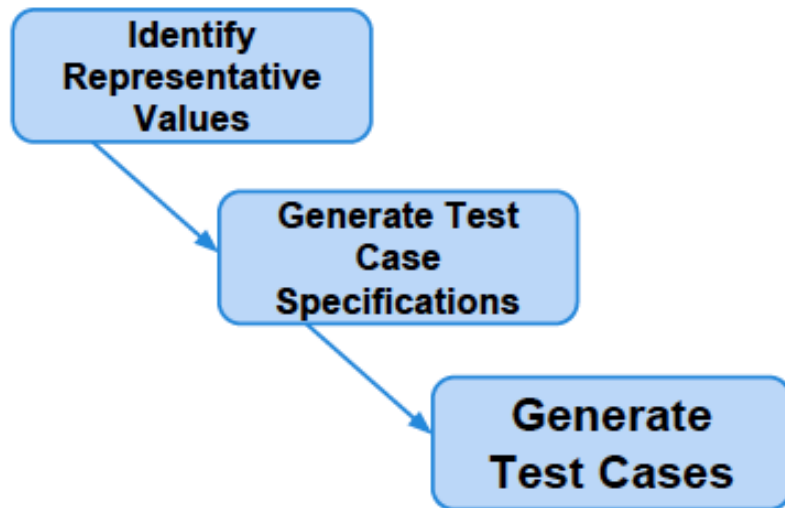
We could consider `a` or `b` in isolation:

`a < 0`, `a = 0`, `a > 0`

We should also consider the combinations of `a` and `b` that influence the outcome of `c`:

`a > b`, `a < b`, `a = b`

Creating Requirements-Based Tests



For each independently testable feature, we want to:

1. Identify the representative value partitions for each input or output.
2. Use the partitions to form abstract test specifications for the combination of inputs.
3. Then, create concrete test cases by assigning concrete values from the set of input partitions chosen for each possible test specification.

Equivalence Partitioning

Feature `insert(int N, list A)`.

Partition inputs into equivalence classes.

1. `int N` is a 5-digit integer between 10000 and 99999.
Possible partitions:
<10000, 10000-99999, >100000
2. `list A` is a list of length 1-10. Possible partitions:
**Empty List, List of Length 1, List of Length 2-10,
List of Length > 10**

Step 4: From Partition to Test Case (Test Case Specifications)

Choose concrete values for each combination of input partitions: `insert(int N, list A)`

`int N`

< 10000

10000 - 99999

> 99999

`list A`

Empty List

List[1]

List[2-10]

List[>10]

Test Specifications:

`insert(< 10000, Empty List)`

`insert(10000 - 99999, list[1])`

`insert(> 99999, list[2-10])`

etc

Test Cases:

`insert(5000, {})`

`insert(96521, {11123})`

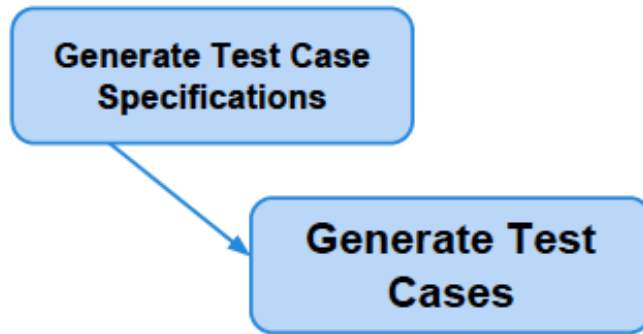
`insert(150000, {11123, 98765})`

etc

Test Specifications – will only showcase input partitions

Test Cases – will only showcase exacts parameters input we will pass into the methods and run the test.

Step 5: Generate Test Cases



```
substr(string  
str, int index)
```

Specification:

`str`: length ≥ 2 , contains
special characters

`index`: value > 0

Test Case:

`str = "ABCC!\n\t7"`

`index = 5`

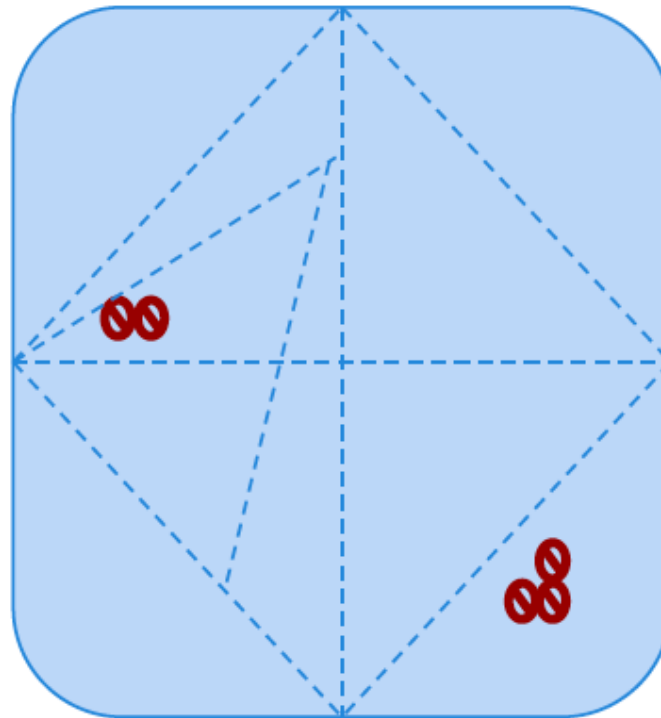
Activity

- Consider a user registration module with the function:
 - User_registration(str First_name, str Last_name, Date dob, str email)
 - Come up with partitions
 - Come up with equivalence classes
 - Come up with test specifications
 - Come up with test cases

Boundary Values

Basic Idea:

- Errors tend to occur at the boundary of a partition.
- Remember to select inputs from those boundaries.

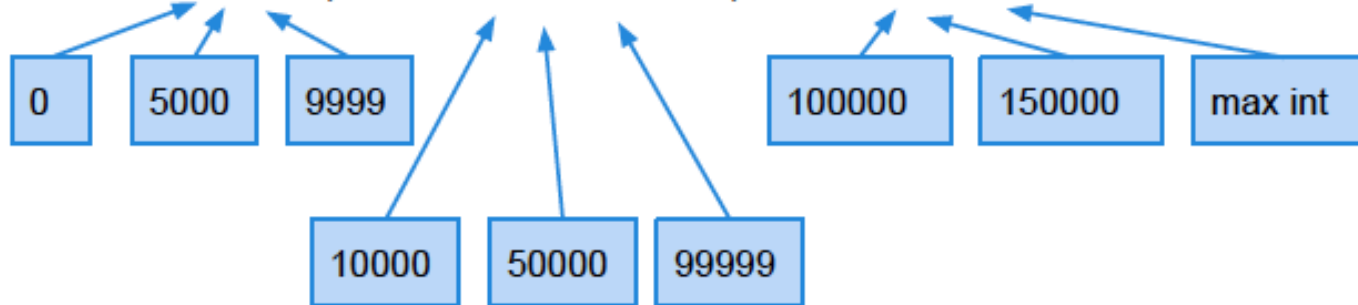


Choosing Test Case Values

Choose test case values at the boundary (and typical) values for each partition.

- If an input is intended to be a 5-digit integer between 10000 and 99999, you want partitions:

<10000, 10000-99999, >100000

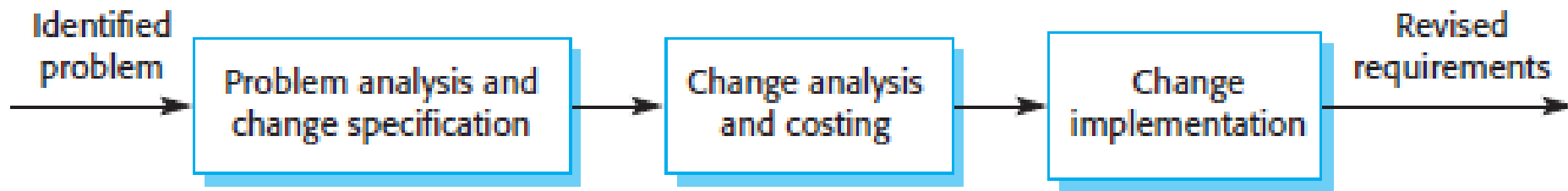


Change management

Requirements management planning

- Requirements Identification
- Change management process
- Traceability policies
- Tool support

Stages of Change management



SRS Tools

SRS tools

- SRS – System Requirements Specifications
- Some tools will help us further improve system requirements
 - Data dictionaries
 - Data Flow Diagrams
 - Process Specifications
 - ER diagrams
 - UML diagrams

Data Dictionaries

- It lists all the data items appearing in Data Flow Diagram.
- File or a set of files that includes a database's metadata (hold records about other objects in the database), like data ownership, relationships of the data to another object, and some other data.
- Data dictionary contains the following information:
 - Name of the item: It can be your choice.
 - Aliases: It represents another name.
 - Description: Description of what the actual text is all about.
 - Related data items: with other data items.
 - Range of values: It will represent all possible answers.

Data Dictionary Notations tables :

Notations	Meaning
$X = a+b$	X consists data elements a and b.
$X = [a/b]$	X consists of either elements a or b.
$X = a \ X$	X consists of optimal data elements a.
$X = y[a]$	X consists of y or more events of data element a
$X = [a] \ z$	X consists of z or less events of data element a
$X = y \ [a] \ z$	X consists of some events of data elements between y and z.

Pros and Cons

PROS

- Consistency and Standardization
- Data Quality
- Data Integration
- Improved collaboration
- Improved Efficiency

CONS

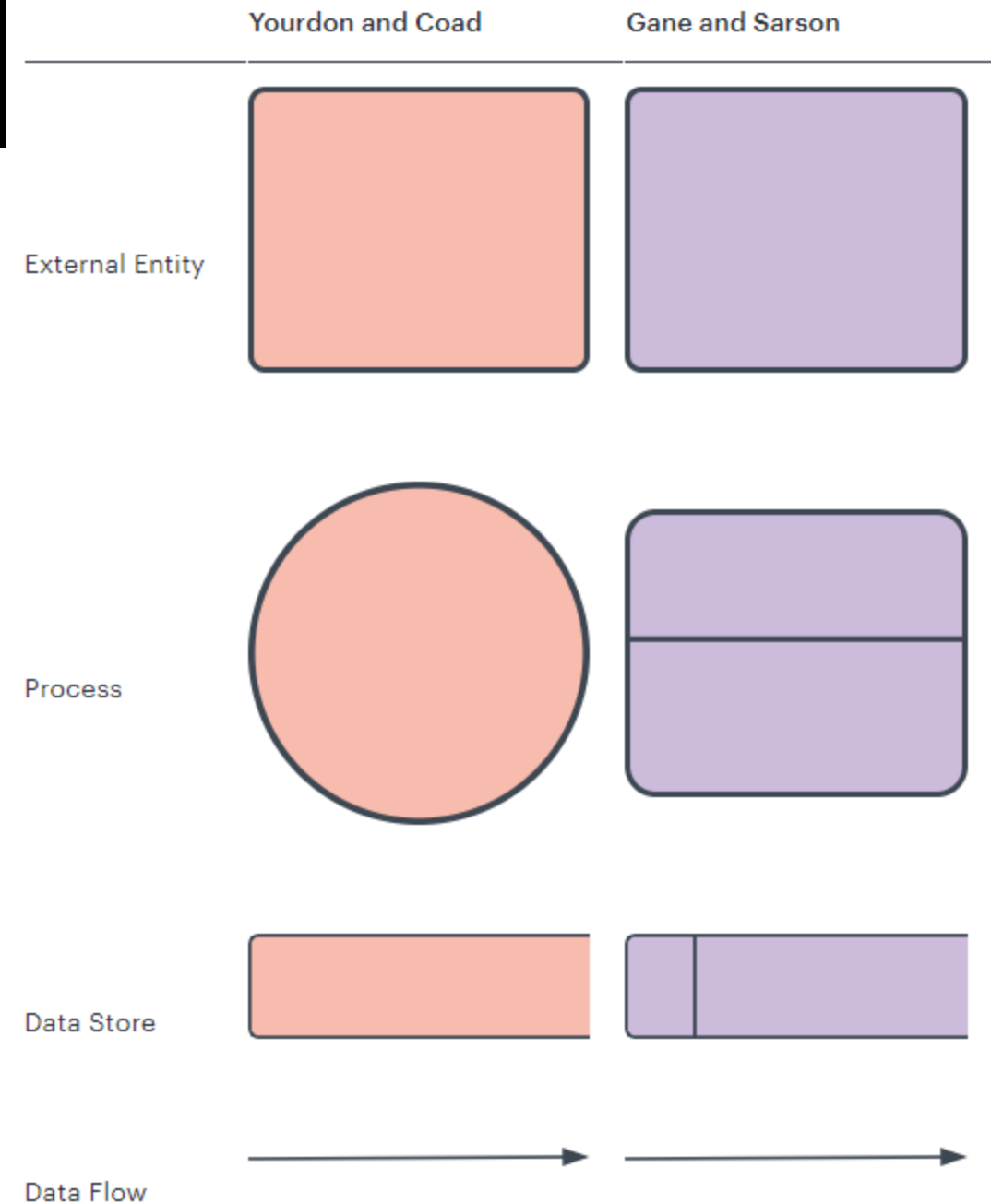
- Implementation and Maintenance Costs
- Complexity
- Resistance to Change
- Data Security
- Data Governance

Data Flow Diagram (DFD)

- A data flow diagram shows the way information flows through a process or system
- It includes data inputs and outputs, data stores, and the various subprocesses the data moves through
- visually represent systems and processes that would be hard to describe in just words.

DFD Symbols and Notations

- External Entities :
 - Actors, sources, sinks
- Process:
 - Activities that transform the data
- Data store:
 - holds data for later access
 - Input flows to a data store include information or operations that change the stored data. Output flows would be data retrieved from the store.
- Data flow
 - Movement of data between external entities, processes and data stores is represented with an arrow symbol, which indicates the direction of flow



Levels of DFD

- **Level 1 :**

- are still a general overview, but they go into more detail than a context diagram
- the single process node from the context diagram is broken down into sub-processes

- **Level 2:**

- simply break processes down into more detailed sub-processes

Reading Assignments

- Section 4.6 of the textbook